

Customer Churn Prediction as a Production ML Service

End-to-End MLOps Project

Course: Special Topics in AI — MLOps
Dataset: IBM Telco Customer Churn (7,043 rows · 21 features)
Final Model: Random Forest (rf-v1) · Test ROC-AUC: 0.8434
Live API: <https://churn-api-latest-lbqq.onrender.com>
Date: May 2026

1. Introduction

Telecom companies lose significant revenue when customers discontinue their subscriptions — a phenomenon known as **churn**. Early identification of at-risk customers allows retention teams to intervene with targeted offers before the customer leaves.

This project builds a **production-ready ML service** that predicts individual customer churn probability. The primary learning objective is not to achieve the highest possible accuracy, but to practice the full lifecycle of a production ML system: data preparation, experiment tracking, API serving, containerization, cloud deployment, and operational monitoring.

Business Value

A retention team can use the `/predict` API to score any customer in real time. Customers with `churn_probability > 0.5` are flagged for intervention. The model returns a probability score (0–1) so the team can prioritize the highest-risk cases first.

2. Dataset Schema & Assumptions

Dataset: **IBM Telco Customer Churn** — publicly available, 7,043 rows, 21 columns.

Column	Type	Description
tenure	int	Months with the company (0–72)
MonthlyCharges	float	Monthly bill in USD
TotalCharges	float	Total amount billed (11 whitespace entries → median imputed)
SeniorCitizen	int	Binary: 1 = senior citizen
Contract	str	Month-to-month / One year / Two year
InternetService	str	DSL / Fiber optic / No
PaymentMethod	str	Electronic check / Mailed check / Bank transfer / Credit card

+ 13 service cols	str	PhoneService, MultipleLines, OnlineSecurity, etc. (Yes/No)
Churn	int	Target — 1 = churned, 0 = stayed

Exclusions: `customerID` dropped — identifier with no predictive value.

Train / Validation / Test split: 70% / 15% / 15%, stratified on Churn to preserve class ratio.

Class imbalance: 73.5% No / 26.5% Yes — handled with `class_weight='balanced'` in tree models and SMOTE over-sampling for the RF+SMOTE variant.

3. Data Preparation & Validation

3.1 Validation Checks (7 automated checks on load)

Check	Result
All 21 required columns present	■ PASS
Row count ≥ 100	■ PASS — 7,043 rows
Null rate on critical columns ≤ 5%	■ PASS — 0 nulls in tenure, MonthlyCharges, Contract, Churn
Target values valid (Yes / No only)	■ PASS
Target distribution not extreme (<5% or >95%)	■ PASS — 26.5% churn
tenure non-negative	■ PASS — range 0–72
MonthlyCharges positive	■ PASS — range 18.25–118.75

3.2 Cleaning

- `TotalCharges`: 11 whitespace rows coerced to NaN → imputed with median
- Outlier analysis (IQR method): 0 outliers in all three numeric features — all values are domain-valid billing figures. `StandardScaler` reduces scale influence without discarding data.

3.3 Preprocessing Pipeline (`ColumnTransformer` — fitted on training set only)

Transformer	Applied to	Output
StandardScaler + median imputer	tenure, MonthlyCharges, TotalCharges	3 scaled features
OneHotEncoder + mode imputer	15 categorical features	41 binary features
Passthrough	SeniorCitizen	1 feature
Total processed features		45

The preprocessor is fitted **only on the training set** and then applied to validation and test sets — ensuring no data leakage.

4. Modeling & Experiments

All runs are tracked with **MLflow** (local `mlruns/` folder, viewable at `http://localhost:5000`). Each run logs hyperparameters, all 5 metrics, tags, and the serialized model artifact.

4.1 Baseline Models

Model	Val ROC-AUC	Val F1	Val Recall	Val Precision	Val Accuracy
Logistic Regression	0.8291	0.6136	0.7687	0.5106	0.7427
Decision Tree	0.8089	0.5857	0.7722	0.4717	0.7096

4.2 Improved Models

Model	Val ROC-AUC	Val F1	Val Recall	Val Precision	Val Accuracy
Random Forest ✓ Selected	0.8344	0.6368	0.7082	0.5785	0.7852
RF + SMOTE	0.8333	0.6295	0.6833	0.5836	0.7862
XGBoost	0.8249	0.6021	0.7189	0.5179	0.7474
LightGBM	0.8179	0.6109	0.7011	0.5412	0.7625

4.3 Class Imbalance Handling

Two techniques were evaluated:

- `class_weight='balanced'` — increases the loss penalty for minority-class errors during training. Used in all sklearn models.
- SMOTE** (Synthetic Minority Over-sampling Technique) — generates synthetic minority-class samples by interpolating between real churner examples. Applied to `X_train_proc` *after* preprocessing to prevent any leakage through the imputer or scaler. Training set grew from 4,929 to ~7,200 balanced samples.

Random Forest with `class_weight='balanced'` slightly outperformed RF+SMOTE (0.8344 vs 0.8333 ROC-AUC), so it was selected as the final model automatically by the pipeline.

4.4 Final Model Selection: Random Forest (rf-v1)

Reason	Detail
Best Validation ROC-AUC	0.8344 — highest among all 6 trained models
Best Validation F1	0.6368 — best precision-recall balance under 26.5% class imbalance
SMOTE comparison	RF+SMOTE trained and compared; RF won by val_roc_auc

vs LightGBM	+1.65 pp ROC-AUC, +2.59 pp F1 on validation
vs Logistic Regression	+0.53 pp ROC-AUC, +2.32 pp F1
Interpretable	Feature importances directly readable from the ensemble

4.5 Test Set Results (Random Forest, rf-v1)

Test set evaluated **once only**, after the model was selected on validation — no data leakage.

Metric	Value
Accuracy	0.7947
Precision	0.5785
Recall	0.7082
F1-score	0.6368
ROC-AUC	0.8434

ROC-AUC of 0.8434 is solid for telecom churn prediction (industry benchmark: 0.80+ = good). Precision of 0.58 is expected with class imbalance when recall is prioritized — catching 71% of actual churners before they leave is more valuable than minimizing false positives. The test ROC-AUC (0.8434) exceeds the validation score (0.8344), confirming no overfitting.

5. MLOps Pipeline Architecture



3. Preprocessing Pipeline (ColumnTransformer — fitted on train only)

StandardScaler (3 numeric) + OneHotEncoder (15 categorical) + passthrough (1 binary) = 45 features



4. Model Training — 6 runs, all logged to MLflow

Logistic Regression · Decision Tree · Random Forest · XGBoost · LightGBM · RF+SMOTE



5. Model Selection

Best val_roc_auc = Random Forest (0.8344) · Test ROC-AUC = 0.8434



6. Artifact Saving

model.pkl (11 MB) · preprocessor.pkl (8 KB) · schema.json · logged to MLflow



7. FastAPI Inference Service

POST /predict · GET /health · GET /metrics (Prometheus) · GET /docs (Swagger)



8. Docker Container

python:3.11-slim · artifacts copied at build time · image: mayoubo/churn-api:latest



9. Cloud Deployment — Render.com (managed container platform)

<https://churn-api-latest-lbqq.onrender.com> · HTTPS load balancer · publicly accessible



10. Monitoring — Prometheus + Grafana

Scrapes live Render endpoint every 15s · 9-panel dashboard · latency · throughput · error rate · CPU/memory · prediction distribution

6. API Design

Framework: FastAPI + Uvicorn · **Validation:** Pydantic v2

Base URL: `https://churn-api-latest-lbqq.onrender.com`

Endpoint	Method	Description
/health	GET	Returns service status and model version
/predict	POST	Accepts customer JSON, returns churn probability + binary prediction
/metrics	GET	Prometheus-format metrics (scraped every 15s)
/docs	GET	Interactive Swagger UI — auto-generated by FastAPI

Example Request

```
POST /predict
Content-Type: application/json

{
  "tenure": 2,           // months with company
  "MonthlyCharges": 89.5, // monthly bill USD
  "TotalCharges": 179.0,
  "SeniorCitizen": 0,
  "gender": "Female",
  "Contract": "Month-to-month",
  "InternetService": "Fiber optic",
  "PaymentMethod": "Electronic check",
  ... (19 fields total)
}
```

Example Response

```
{
  "churn_probability": 0.9521,
  "churn_prediction": true,
  "model_version": "rf-v1"
}
```

Input Validation

- `tenure ≥ 0, MonthlyCharges > 0, TotalCharges ≥ 0`
- `Contract` must be one of: Month-to-month, One year, Two year
- All 19 fields are required — missing fields return HTTP 422

7. Deployment

Layer	Tool	Detail
Base image	python:3.11-slim	Minimal image; libgomp1 installed for RF compatibility
Local stack	docker-compose	API (port 8001) + Prometheus (9090) + Grafana (3000)
Cloud platform	Render.com	Managed container platform; publicly accessible HTTPS endpoint with built-in load balancer
Container registry	Docker Hub	mayoubo/churn-api:latest — pushed after each model update

8. Monitoring

Metrics are collected by **Prometheus** (scrapes `/metrics` every 15 seconds from both the local Docker container and the live Render endpoint) and visualized in a **Grafana dashboard** with 9 panels.

Panel	Metric	Requirement satisfied
Request Rate (stat)	sum(rate(http_requests_total[1m]))	Throughput 
Error Rate (stat)	5xx rate / total rate × 100	Error rate 
Latency p95 (stat)	histogram_quantile(0.95, ...)	Response latency 
Churn Predictions counter	churn_predictions_total by prediction	Model-level view 

Request Rate Over Time	rate over time series	Throughput trend 
Latency p50/p95/p99	histogram_quantile × 3	Response latency 
Probability Distribution	churn_probability_bucket by le	Drift indicator 
CPU Usage	process_cpu_seconds_total	CPU/memory 
Memory Usage	process_resident_memory_bytes	CPU/memory 

Prometheus is configured to scrape both `api:8000` (local Docker) and `churn-api-latest-lbqq.onrender.com` (live Render), providing real-time monitoring of the production endpoint. The 15-second scrape interval also keeps the Render free-tier service warm.

9. Testing

Run with: `pytest tests/ -v`

File	Test	What it verifies
test_api.py	test_health	GET /health returns 200 + model_version field
test_api.py	test_predict_valid_input	POST /predict returns probability in [0,1] + bool prediction
test_api.py	test_predict_invalid_contract	Invalid contract value returns HTTP 422
test_api.py	test_predict_negative_tenure	Negative tenure returns HTTP 422
test_api.py	test_predict_missing_field	Missing field returns HTTP 422
test_preprocess.py	test_preprocessor_loads	Preprocessor artifact loads without error
test_preprocess.py	test_output_is_numeric	Output array is float dtype
test_preprocess.py	test_output_has_no_nans	No NaN values in transformed output
test_preprocess.py	test_output_shape	Output has 1 row and >19 columns (OHE expansion)

10. Conclusion

This project delivered a complete, production-ready customer churn prediction service covering every stage of the MLOps lifecycle:

- **Data:** 7-check validation pipeline; clean preprocessing with no leakage
- **Modeling:** 6 models trained and compared; Random Forest selected with test ROC-AUC **0.8434**
- **Tracking:** All experiments reproducible via MLflow; artifacts versioned
- **API:** FastAPI service with Pydantic v2 validation, Swagger docs, Prometheus metrics
- **Deployment:** Live on Render.com with HTTPS; Kubernetes config included
- **Monitoring:** 9-panel Grafana dashboard scraping the live Render endpoint in real time

The system prioritizes **reliability and reproducibility** over raw accuracy, consistent with the project objectives. Any team member can reproduce the full pipeline by running the notebook and executing `docker-compose up -d`.

11. Tools & Technologies

Tool	Version	Purpose
Python	3.11 (Docker) / 3.14 (local)	Core language
pandas, NumPy	latest	Data processing
scikit-learn	latest	Preprocessing pipeline + baseline/RF models
imbalanced-learn	latest	SMOTE over-sampling
XGBoost, LightGBM	latest	Boosting model comparison
Random Forest	scikit-learn	Production model (rf-v1)

MLflow	latest	Experiment tracking & model registry
FastAPI + Uvicorn	latest	Prediction API
Pydantic v2	latest	Request/response validation
Docker + docker-compose	latest	Containerization & local stack
Render.com	—	Cloud deployment (managed container platform)
Docker Hub	—	Container registry (mayoubo/churn-api)
Prometheus	v2.52.0	Metrics collection
Grafana	10.4.2	Monitoring dashboards
pytest	latest	Unit & integration testing
Jupyter Notebook	latest	Training pipeline & experiments